

Question 1: What is Matplotlib, and why is it used for data visualization in Python? Explain the difference between plt.plot() and plt.scatter() with suitable examples.

Answer - **Matplotlib** is a comprehensive, open-source library for creating static, animated, and interactive visualizations in Python. It provides an object-oriented API for embedding plots into applications and closely emulates the plotting behavior of MATLAB.

It is widely used in data science, machine learning, and research for several key reasons:

- **Data Exploration & Insights:** Helps analysts quickly visualize trends, outliers, and distributions in raw data.
- **Customization:** Offers granular control over every element of a figure, including line styles, colors, font sizes, axes limits, and labels.
- **Publication-Quality Output:** Can export figures in various high-resolution formats (PNG, PDF, SVG, EPS) suitable for research papers and reports.
- **Integration:** Seamlessly integrates with data manipulation libraries like **NumPy** and **Pandas**.

2. Difference Between **plt.plot()** and **plt.scatter()**

Feature	plt.plot()	plt.scatter()

Primary Use	Used to create line plots to show trends over continuous intervals or time series.	Used to create scatter plots to show the correlation or relationship between two distinct variables.
Data Representation	Connects individual data points with line segments (though markers can also be enabled).	Displays individual data points as independent markers (dots, circles, squares) without connecting lines.
Performance	Faster when rendering large datasets with lines because fewer discrete objects are drawn.	Can be slower with massive datasets because every marker is plotted individually, but allows independent control over each point's size and color.

3. Code Examples

Example 1: Using `plt.plot()` (Line Plot)

`plt.plot()` is ideal for tracking continuous changes, such as stock prices or temperature over time.

Python

```
import matplotlib.pyplot as plt
```

```
# Days of the week and corresponding temperatures
```

```
days = ["Mon", "Tue", "Wed", "Thu", "Fri"]
```

```
temperatures = [28, 30, 27, 32, 31]
```

```
# Create a line plot
```

```
plt.plot(days, temperatures, marker="o", color="blue", linestyle="-")
```

```
plt.title("Weekly Temperature Trends")
```

```
plt.xlabel("Days")
```

```
plt.ylabel("Temperature (°C)")
```

```
plt.grid(True)
```

```
plt.show()
```

Example 2: Using `plt.scatter()` (Scatter Plot)

`plt.scatter()` is ideal for analyzing relationships, such as the correlation between student study hours and exam scores.

Python

```
import matplotlib.pyplot as plt
```

```
# Study hours and corresponding exam scores
```

```
study_hours = [1.5, 2.0, 3.5, 4.0, 5.0, 6.5]
```

```
exam_scores = [55, 60, 72, 78, 88, 95]
```

```
# Create a scatter plot
```

```
plt.scatter(
```

```
    study_hours, exam_scores, color="red", s=100
```

```
) # 's' controls marker size
```

```
plt.title("Study Hours vs. Exam Scores")
```

```
plt.xlabel("Hours Studied")
```

```
plt.ylabel("Exam Score")
```

```
plt.grid(True)
```

```
plt.show()
```

Question 2: What is a Figure and an Axes in Matplotlib? Explain the difference between `plt.subplot()` and `plt.subplots()` with an example.

Answer - In Matplotlib, the visualization hierarchy is divided into container objects and element objects. Understanding the precise distinction between a **Figure** and an **Axes** is fundamental to using the object-oriented API effectively:

- **Figure (The Top-Level Container):**
 - A **Figure** object is the overall window, canvas, or top-level container that holds everything in a given plot or graphic.
 - It can house multiple subplots, overall titles, legends, and colorbars.
 - Think of the Figure as the entire blank sheet of paper, drawing board, or window pane.
- **Axes (The Plot Area / Subplot):**
 - An **Axes** object represents an individual plot or graph area *inside* a Figure.
 - It contains the actual data space, X and Y axis lines, tick marks, labels, titles, and data points (such as lines, bars, or scatter points).
 - A single Figure can contain one or multiple Axes objects.
(Note: *Axes is distinct from "Axis", which refers specifically to the individual X or Y number lines*).

2. Difference Between `plt.subplot()` and `plt.subplots()`

Both functions are used to create subplots, but they differ significantly in their syntax, workflow, and flexibility:

Feature / Property	<code>plt.subplot()</code>	<code>plt.subplots()</code>

Syntax & Return Value	Creates and returns a single Axes object at a time based on grid position (nrows , ncols , index).	Creates and returns both the entire Figure object and an array/matrix of Axes objects all at once.
Workflow Approach	State-based / procedural approach. Often requires calling subplot indexing repeatedly to add multiple plots.	Object-oriented approach. Highly recommended for multi-panel figures as it gives direct access to all axes upfront.
Flexibility in Layouts	Less convenient when modifying global figure attributes or sharing axes across complex grids.	Extremely flexible. Easily handles shared axes (sharex=True , sharey=True) and complex grid arrangements in a single line.
Best Used For	Quick interactive scripts or adding individual subplots sequentially.	Production-grade code, multi-panel dashboards, and clean object-oriented data visualizations.

3. Code Example Using `plt.subplots()`

The following complete Python script demonstrates how to use `plt.subplots()` to create a **1x2** grid of subplots (two side-by-side graphs) sharing the same figure container:

Python

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
# Generate sample data
```

```
x = np.linspace(0, 10, 100)
```

```
y1 = np.sin(x)
```

```
y2 = np.cos(x)
```

```
# Create a figure and a grid of 1 row by 2 columns of Axes
```

```
fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(10, 4))
```

```
# Plot on the first axes (ax1)
```

```
ax1.plot(x, y1, color='blue', linewidth=2)
```

```
ax1.set_title('Sine Wave')
```

```
ax1.set_xlabel('X values')
```

```
ax1.set_ylabel('Sin(x)')
```

```
ax1.grid(True, linestyle='--')
```

```
# Plot on the second axes (ax2)
```

```
ax2.plot(x, y2, color='red', linewidth=2)
```

```
ax2.set_title('Cosine Wave')
```

```
ax2.set_xlabel('X values')
```

```
ax2.set_ylabel('Cos(x)')
```

```
ax2.grid(True, linestyle='--')
```

```
# Adjust layout and set a global title for the figure
```

```
fig.suptitle('Comparison of Sine and Cosine Waves', fontsize=14,  
fontweight='bold')
```

```
plt.tight_layout()
```

```
plt.show()
```

Question 3: Explain how to customize a line plot in Matplotlib using color, linestyle, linewidth, marker, and label. Write a Python program to plot a sine wave using these customizations.

Answer - Matplotlib provides extensive styling options to enhance the readability, visual appeal, and clarity of line plots. Below is a detailed breakdown of the five key customization parameters:

- **color** (Line Color): Controls the color of the plotted line. Matplotlib accepts standard color names (e.g., 'blue', 'red'), single-letter shorthands ('b', 'r'), and hex color codes (e.g., '#ff7f0e', '#1f77b4').
- **linestyle** (Line Style / Pattern): Defines the visual pattern of the line. Common options include solid lines ('-' or 'solid'), dashed lines ('--' or 'dashed'), dotted lines (':' or 'dotted'), and dash-dot lines ('-.' or 'dashdot').
- **linewidth** (Line Thickness): Specifies the thickness of the line in points. Higher values create bolder, more prominent lines (e.g., `linewidth=2.5`), while lower values create thinner lines.
- **marker** (Data Point Markers): Places geometric symbols at each individual data point coordinate along the line. Popular marker styles include circles ('o'), squares ('s'), triangles ('^'), and stars (*).
- **label** (Legend Label): Assigns a descriptive text string to the plot line, which is displayed when `plt.legend()` is called. This is crucial for identifying distinct lines on the axes.

2. Python Program to Plot a Sine Wave with Customizations

The following complete, production-ready Python program utilizes NumPy to generate sine wave data and Matplotlib to plot it incorporating all the specified customizations:

Python

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
# Step 1: Generate X values from 0 to 10 and compute  
corresponding sine values
```

```
x = np.linspace(0, 10, 100)
```

```
y = np.sin(x)
```

```
# Step 2: Initialize figure and axes using the object-oriented API
```

```
fig, ax = plt.subplots(figsize=(9, 5))
```

```
# Step 3: Plot sine wave using color, linestyle, linewidth, marker,  
and label
```

```
ax.plot(
```

```
    x, y,
```

```
    color='#1f77b4',          # Custom hex color (Blue)
```

```
    linestyle='--',          # Dashed line style
```

```
    linewidth=2.5,           # Line thickness of 2.5 points
```

```
marker='o',          # Circle marker at data points

markevery=5,          # Display marker on every 5th point to
avoid clutter

markersize=6,         # Marker size

label='Sine Wave (sin(x))' # Legend label

)
```

Step 4: Add titles, labels, grid, and legend for complete presentation

```
ax.set_title('Customized Sine Wave Plot', fontsize=14,
fontweight='bold', pad=15)
```

```
ax.set_xlabel('X Axis (Radians)', fontsize=12)
```

```
ax.set_ylabel('Y Axis Amplitude', fontsize=12)
```

```
ax.grid(True, linestyle=':', alpha=0.7)
```

```
ax.axhline(0, color='black', linewidth=1, alpha=0.5) # X-axis
baseline
```

```
ax.legend(loc='upper right', fontsize=11)
```

```
plt.tight_layout()
```

Step 5: Display the plot

plt.show()

Feature / Dimension	Matplotlib[cite: 1]	Seaborn[cite: 1]

Level of Abstraction	Low-level plotting library. Gives granular control over individual lines, axes, and canvas pixels[cite: 1].	High-level interface. Abstracts away complex boilerplate code for statistical plotting[cite: 1].
Data Integration	Works primarily with NumPy arrays, lists, and basic numerical sequences[cite: 1].	Designed natively to work with Pandas DataFrames, utilizing column names directly as variables[cite: 1].
Statistical Aggregation	Does not compute statistical summaries automatically; requires manual data pre-processing[cite: 1].	Automatically performs statistical estimations (e.g., calculating means, confidence intervals, regression lines)[cite: 1].
Aesthetics & Default Themes	Basic default styling; requires extensive manual configuration to look modern and polished[cite: 1].	Comes with beautiful, modern built-in themes and color palettes out-of-the-box[cite: 1].

Multi-Variar ble Data	Requires writing loops, custom subplots, and manual filtering to display multi-dimensional relationships[cite: 1].	Easily handles complex multi-variable relationships using built-in parameters like hue , col , and row [cite: 1].
----------------------------------	--	--

Question 4: What is Seaborn, and how is it different from Matplotlib? Explain the concept of hue in Seaborn plots with a suitable example.

Answer - **Seaborn** is a high-level Python data visualization library built on top of Matplotlib.

- It is tightly integrated with Pandas data structures and is specifically designed for drawing attractive, informative, and complex statistical graphics.
- While Matplotlib provides low-level building blocks and granular control over every individual pixel, Seaborn acts as a high-level wrapper that automates much of the complex statistical aggregation, color mapping, and styling required to turn raw data into insightful visualizations with minimal code[cite: 1].

2. Differences Between Seaborn and Matplotlib

Although Seaborn relies on Matplotlib under the hood, they differ significantly in focus, syntax, and built-in capabilities[cite: 1]:3. The Concept of "hue" in Seaborn

- The **hue** parameter in Seaborn is a powerful feature used to introduce a third variable into a two-dimensional plot by mapping it to distinct colors[cite: 1].
- Instead of creating separate subplots or filtering data manually, passing a categorical or numerical column to **hue** allows Seaborn to automatically split and color-code the visual markers, bars, or lines based on unique categories within that variable[cite: 1].
- This enables analysts to visually inspect interactions, groupings, and subgroup trends (e.g., comparing data across gender, regions, or classes) within a single unified figure[cite: 1].

4. Python Example Using **hue** in a Seaborn Scatter Plot

The following complete Python program demonstrates the use of Seaborn's **scatterplot()** function, utilizing the **hue** parameter to differentiate data points by category (e.g., smoker vs. non-smoker status in a built-in **tips** dataset)[cite: 1]:

Python

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# Load built-in sample dataset from Seaborn
```

```
tips = sns.load_dataset('tips')
```

```
# Initialize the figure canvas using Matplotlib
```

```
plt.figure(figsize=(9, 6))
```

```
# Create a scatter plot showing total bill vs. tip amount,
```

```
# with 'smoker' category mapped to distinct colors via the 'hue'  
parameter
```

```
sns.scatterplot(  
    data=tips,  
    x='total_bill',  
    y='tip',  
    hue='smoker',  
    palette='magma',  
    s=100,  
    legend=True,  
)
```

```
y='tip',  
  
hue='smoker',    # Third variable mapped to colors  
  
style='smoker',  # Optional: different markers for each category  
  
palette='Set1',  # Color palette  
  
s=100,          # Marker size  
  
alpha=0.8  
  
)  
  
# Add titles and formatting  
  
plt.title('Total Bill vs. Tip Amount Grouped by Smoker Status (Hue)',  
fontsize=14, fontweight='bold', pad=15)  
  
plt.xlabel('Total Bill ($)', fontsize=12)  
  
plt.ylabel('Tip Amount ($)', fontsize=12)  
  
plt.grid(True, linestyle='--', alpha=0.5)  
  
  
plt.legend(title='Smoker Status', loc='upper left')  
  
plt.tight_layout()  
  
plt.show()
```

Question 5: Explain the difference between `plt.bar()` and `plt.barh()`. Consider the following data: `categories = ["A", "B", "C", "D"]` `values = [23, 17, 35, 29]` Write a Python program to create both a vertical and a horizontal bar chart using this data.

Answer - 1. Difference Between `plt.bar()` and `plt.barh()`

- `plt.bar()`: Used to create a vertical bar chart. The categories are displayed along the x-axis, and their corresponding heights/values are represented along the y-axis. It is best used when comparing discrete categories with relatively short names.
- `plt.barh()`: Used to create a horizontal bar chart. The categories are displayed along the y-axis, and their corresponding lengths/values extend along the x-axis. It is ideal when category labels are long, or when you have a large number of categories, as it prevents text overlapping on the axis.

2. Python Program for Both Charts

Python

```
import matplotlib.pyplot as plt
```

```
# Given data
```

```
categories = ["A", "B", "C", "D"]
```

```
values = [23, 17, 35, 29]
```

```
# Create a figure with two subplots (1 row, 2 columns)
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
```

1. Vertical Bar Chart (plt.bar)

```
ax1.bar(categories, values, color='skyblue', edgecolor='black',  
width=0.6)
```

```
ax1.set_title('Vertical Bar Chart (plt.bar)', fontsize=14,  
fontweight='bold')
```

```
ax1.set_xlabel('Categories', fontsize=12)
```

```
ax1.set_ylabel('Values', fontsize=12)
```

```
ax1.grid(axis='y', linestyle='--', alpha=0.7)
```

2. Horizontal Bar Chart (plt.barh)

```
ax2.barh(categories, values, color='salmon', edgecolor='black',  
height=0.6)
```

```
ax2.set_title('Horizontal Bar Chart (plt.barh)', fontsize=14,  
fontweight='bold')
```

```
ax2.set_xlabel('Values', fontsize=12)
```

```
ax2.set_ylabel('Categories', fontsize=12)
```

```
ax2.grid(axis='x', linestyle='--', alpha=0.7)
```

Adjust layout and display the plot

plt.tight_layout()

plt.show()

Question 6: What is a histogram, and what does its shape tell us about a dataset? Write a Python program to generate 1000 random values using `np.random.randn()` and plot a histogram with `bins=30`.

Answer - . What is a Histogram?

- **A histogram is a graphical representation that organizes a group of numerical data points into specified continuous ranges, known as "bins".**
- **While it resembles a vertical bar chart, the x-axis represents continuous numerical intervals rather than discrete categories, and the y-axis represents the frequency (count) of data points falling within each interval.**

2. What the Shape of a Histogram Tells Us

The overall shape of a histogram reveals crucial insights about the underlying distribution of a dataset:

Histogram Shape / Pattern	Statistical Meaning & Dataset Characteristics

Normal / Bell-shaped Distribution	Symmetric shape where data clusters evenly around a central peak, tapering off equally on both sides. Indicates that the mean, median, and mode are roughly equal.
Right-Skewed (Positively Skewed)	Long tail extends to the right side, meaning most data values are concentrated on the left with a few exceptionally high values (e.g., income or housing prices).
Left-Skewed (Negatively Skewed)	Long tail extends to the left side, indicating that most data values are concentrated on the higher end with a few exceptionally low values.
Uniform Distribution	Flat or rectangular shape where all intervals have approximately equal frequencies, indicating that data values are spread evenly across the entire range.

3. Python Program for Question 6

Python

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
# Set random seed for reproducibility
```

```
np.random.seed(42)
```

```
# Generate 1000 random values from a standard normal  
distribution
```

```
data = np.random.randn(1000)
```

```
# Initialize the figure canvas
```

```
plt.figure(figsize=(9, 6))
```

```
# Plot the histogram with 30 bins
```

```
count, bins, ignored = plt.hist(
```

```
    data,
```

```
    bins=30,
```

```
color='skyblue',

edgecolor='black',

alpha=0.85

)


# Add titles, labels, and formatting

plt.title('Histogram of 1000 Random Values (Standard Normal
Distribution)', fontsize=14, fontweight='bold', pad=15)

plt.xlabel('Value Intervals (Bins)', fontsize=12)

plt.ylabel('Frequency (Count)', fontsize=12)

plt.grid(axis='y', linestyle='--', alpha=0.6)


# Display statistical annotations

plt.axvline(data.mean(), color='red', linestyle='dashed',
linewidth=1.5, label=f'Mean: {data.mean():.2f}')


plt.legend(loc='upper right')

plt.tight_layout()

plt.show()
```

Question 7: Consider the following dataset: sizes = [40, 25, 20, 15] labels = ["Apples", "Bananas", "Cherries", "Dates"] Write a Python program to: 1. Create a pie chart showing the percentage share of each fruit. 2. Display the percentages on the chart using autopct. 3. Set the start angle of the pie chart to 120 degrees. 4. Explain why plt.axis("equal") is used in a pie chart.

Answer - 1. Explanation of `plt.axis("equal")`

- In Matplotlib, `plt.axis("equal")` (or `plt.gca().set_aspect("equal")`) is used to enforce an equal aspect ratio between the x-axis and y-axis.
- By default, Matplotlib fits the plot into a rectangular window, which can distort circles (like pie charts) into ovals.
- Using `plt.axis("equal")` ensures that the pie chart is rendered as a perfect, symmetrical circle.

2. Python Program for Question 7

Python

```
import matplotlib.pyplot as plt
```

```
# Given dataset
```

```
sizes = [40, 25, 20, 15]
```

```
labels = ["Apples", "Bananas", "Cherries", "Dates"]
```

```
# Initialize the figure canvas
```

```
plt.figure(figsize=(8, 8))

# Define colors and explode for better visual separation

colors = ['#ff9999', '#66b3ff', '#99ff99', '#ffcc99']

explode = (0.05, 0, 0, 0) # slightly explode the 1st slice (Apples)


# Create the pie chart

plt.pie(

    sizes,

    labels=labels,

    autopct='%1.1f%%', # Display percentages with 1 decimal
place

    startangle=120, # Set start angle to 120 degrees

    colors=colors,

    explode=explode,

    shadow=True, # Add shadow effect

    wedgeprops={'edgecolor': 'black', 'linewidth': 1}

)
```

Ensure the pie chart is drawn as a circle

plt.axis('equal')

Add title

**plt.title('Percentage Share of Each Fruit (Pie Chart)', fontsize=14,
fontweight='bold', pad=20)**

Display the plot

plt.tight_layout()

plt.show()

Question 8: What is the purpose of `plt.annotate()` in Matplotlib? Write a Python program to plot a sine wave and annotate its peak point with an arrow and the label "Peak".

Answer - 1. Purpose of `plt.annotate()`

- The `plt.annotate()` function in Matplotlib is used to add text annotations and descriptive labels to specific data points or regions on a plot.
- Unlike standard `plt.text()`, which places text at fixed coordinates without pointers, `plt.annotate()` allows developers to draw an arrow pointing directly from the text label to a specific target coordinate (`xy`).
- This is essential for highlighting key features such as local maxima, minima, inflection points, or anomalies in a dataset.

2. Python Program for Question 8

Python

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
# Generate x values from 0 to 2*pi and corresponding sine wave values
```

```
x = np.linspace(0, 2 * np.pi, 500)
```

```
y = np.sin(x)
```

```
# Initialize the figure canvas
```

```
plt.figure(figsize=(9, 5))
```

```
# Plot the sine wave
```

```
plt.plot(x, y, label='Sine Wave', color='blue', linewidth=2)
```

```
# Identify the peak point (for  $\sin(x)$ , the first peak occurs at  $x = \pi/2$ ,  $y = 1$ )
```

```
peak_x = np.pi / 2
```

```
peak_y = 1.0
```

```
# Annotate the peak point with an arrow and label
```

```
plt.annotate(
```

```
    'Peak',
```

```
    xy=(peak_x, peak_y),          # Point to annotate (the peak)
```

```
    xytext=(peak_x + 0.8, peak_y - 0.3), # Position of the text label
```

```
    arrowprops=dict(
```

```
        facecolor='red',
```

```
        edgecolor='red',
```

```
        shrink=0.05,  
  
        width=1.5,  
  
        headwidth=8  
  
    ),  
  
    fontsize=12,  
  
    fontweight='bold',  
  
    color='red'  
  
)  
  
# Add titles, labels, grid, and legend  
  
plt.title('Sine Wave with Peak Annotation', fontsize=14,  
fontweight='bold', pad=15)  
  
plt.xlabel('X (radians)', fontsize=12)  
  
plt.ylabel('Y = sin(X)', fontsize=12)  
  
plt.axhline(0, color='black', linewidth=0.8, linestyle='--')  
  
plt.grid(True, linestyle='--', alpha=0.6)  
  
plt.legend(loc='lower left')  
  
plt.tight_layout()
```

```
plt.show()
```

Question 9: Consider the following list of scores: scores = [45, 67, 89, 23, 56, 78, 90, 34] Write a Python program using Seaborn to: 1. Load this list into a DataFrame with the column name "Score". 2. Create a histogram of the scores using sns.histplot(). 3. Create a boxplot of the scores using sns.boxplot(). 4. Identify any outliers visible in the boxplot.

Answer - Python

```
import pandas as pd
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
# Given list of scores
```

```
scores = [45, 67, 89, 23, 56, 78, 90, 34]
```

```
# 1. Load the list into a Pandas DataFrame with the column name  
"Score"
```

```
df = pd.DataFrame(scores, columns=['Score'])
```

```
print("DataFrame:")
```

```
print(df)
```

Create a figure with two subplots side-by-side

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
```

2. Create a histogram of the scores using sns.histplot()

```
sns.histplot(data=df, x='Score', kde=True, bins=5, ax=axes[0],  
color='skyblue', edgecolor='black')
```

```
axes[0].set_title('Histogram of Scores with KDE', fontsize=14,  
fontweight='bold')
```

```
axes[0].set_xlabel('Score', fontsize=12)
```

```
axes[0].set_ylabel('Frequency', fontsize=12)
```

```
axes[0].grid(axis='y', linestyle='--', alpha=0.6)
```

3. Create a boxplot of the scores using sns.boxplot()

```
sns.boxplot(data=df, y='Score', ax=axes[1], color='lightgreen')
```

```
axes[1].set_title('Boxplot of Scores', fontsize=14,  
fontweight='bold')
```

```
axes[1].set_ylabel('Score', fontsize=12)
```

```
axes[1].grid(axis='y', linestyle='--', alpha=0.6)
```

`plt.tight_layout()`

`plt.show()`

4. Identification of Outliers

- **Analysis:** Based on the Interquartile Range (IQR) method used in boxplots, the lower and upper whiskers encompass all normal variations within the dataset bounds.
- **Result:** For this specific dataset (`[45, 67, 89, 23, 56, 78, 90, 34]`), all values fall comfortably within the normal range, meaning there are no outliers visible in the boxplot.

Question 10: What is Plotly, and how is it different from Matplotlib and Seaborn? Write a Python program using plotly.express to create an interactive.

Answer - What is Plotly?

- **Plotly** is a high-level, open-source data visualization library built for Python, R, and JavaScript.
- It is designed specifically for creating interactive, publication-quality graphs that can be viewed in web browsers or embedded in web applications. Unlike traditional static plotting libraries, Plotly charts support interactive features such as zooming, panning, hover tooltips, and toggling data series.

2. Differences Between Plotly, Matplotlib, and Seaborn

Feature	Matplotlib	Seaborn	Plotly
Interactivity	Static images by default; requires complex configuration for basic interactivity.	Static by default; built on top of Matplotlib with modern styling.	Fully interactive by default (hover tooltips, zooming, panning out of the box).

Primary Focus	Low-level control over every individual element and canvas pixel.	Statistical data visualization built on top of Matplotlib for quick, attractive graphs.	Web-ready, interactive dashboards and dynamic visualizations.
Output Format	PNG, PDF, SVG (Static raster/vector formats).	PNG, PDF, SVG (Static raster/vector formats).	HTML, JSON, and interactive browser-renderable formats.

3. Python Program Using `plotly.express`

Python

```
import plotly.express as px
```

```
import pandas as pd
```

```
# Given data
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [10, 14, 22, 18, 27]
```

```
# Create a DataFrame for Plotly Express
```

```
df = pd.DataFrame({'X_Value': x, 'Y_Value': y})
```

```
# Create an interactive scatter plot
```

```
fig = px.scatter(
```

```
    df,
```

```
    x='X_Value',
```

```
    y='Y_Value',
```

```
    title='Interactive Scatter Plot using Plotly Express',
```

```
labels={'X_Value': 'X Axis', 'Y_Value': 'Y Axis'},  
  
template='plotly_white'  
  
)  
  
# Customize marker size and color  
  
fig.update_traces(marker=dict(size=12, color='royalblue',  
line=dict(width=2, color='DarkSlateGrey')))  
  
# Display the interactive plot in the browser  
  
fig.show()
```

4. Advantage of Using Plotly Over Static Plots

- **Interactive Hover Tooltips & Zooming:** One major advantage of Plotly over static libraries (like Matplotlib and Seaborn) is that it provides rich interactivity out-of-the-box. When you hover your mouse over any data point, an interactive tooltip instantly reveals its precise (x, y) coordinate values. Static plots require custom coding to achieve similar behavior.

e scatter plot for the following data, and explain one advantage of using Plotly over static plots. $x = [1, 2, 3, 4, 5]$ $y = [10, 14, 22, 18,$